Logistics Operations Platform – Software Requirements

This document describes what the Logistics Operations Platform should do, based on discussions with business users.

It is written in simple language so developers, testers, and stakeholders can easily understand how the system is

expected to behave in real-life operations.

1. Background & Problem

The company currently manages logistics operations using emails, phone calls, WhatsApp messages, and spreadsheets.

There is no single place to see what is happening with an order.

The biggest problems today are:

- No clear visibility of where a shipment is without calling multiple people

- No consistent way to record delays, damages, or failed deliveries

- Too much manual re-entry of the same data

- Billing disputes due to lack of delivery proof or history

The goal of this system is to provide visibility, accountability, and a reliable trail of what actually happened

to each shipment.

2. Who Uses the System

Sales / Account Managers:

- Create orders quickly
- Update orders when customers change details
- Mostly care about speed

Branch Admins / Ops Coordinators:

- Main daily users
- Assign pickups and deliveries
- Track issues and delays

Warehouse Supervisors:

- Confirm goods received and dispatched
- Report damages or shortages
- Usually work from shared computers

Drivers:

- Use the system from personal phones
- Confirm pickups and deliveries
- Upload photos or signatures

Customer Support:

- Look up shipment status when customers call
- Do not change data

Finance / Billing:

- Use the system after delivery
- Verify what was delivered and what should be billed

Management:

- View dashboards and reports
- Not daily users

3. How Orders and Shipments Work

Orders usually come in through emails, calls, or WhatsApp.

Orders are often created with incomplete information to save time.

Details may change later.

A typical flow looks like:

- Order is created
- Ops assigns it to a branch or warehouse
- Pickup is planned and executed
- Shipment enters warehouse
- Shipment moves between hubs if needed
- Out for delivery
- Delivered, partially delivered, or failed
- Billing happens later

This flow is not always clean and may loop or change.

4. What the System Must Do

Order Management:

- Allow creating orders with partial information
- Allow updating orders after creation
- Track all changes made to an order

Shipment Tracking:

- Show current shipment status clearly
- Allow ops to see all shipments across cities

Handoffs & Proof:

- Record when custody changes (warehouse to driver, etc.)
- Capture timestamp and person responsible
- Support photo or signature uploads

Exception Handling:

- Allow recording delays, damages, shortages, failed deliveries
- Keep a clear history of what happened

Customer Updates:

- Send updates at key stages
- Notify customers early when delays happen

Driver & Warehouse Usage:

- Very simple mobile interface
- Minimal typing
- Should work even with poor network

Billing Support:

- Weekly shipment summary for finance
- Clear delivered vs failed vs partial status
- Link delivery proof to billing records

5. Non-Functional Expectations

- System should feel fast and simple
- Drivers should complete actions in very few steps
- Updates should reflect within about 30 seconds
- Data entered should not be lost
- Battery and data usage should be minimal

6. What Success Looks Like

- Ops can answer “where is my shipment?” without phone calls
- Exceptions are visible and consistent
- Customers are informed before they complain
- Finance can close billing without chasing ops

LAB-3

Domain Model

The domain model describes the key conceptual entities involved in the Logistics Operations Platform and the relationships between them. It captures the primary objects that exist in the logistics domain and how they interact during order processing, shipment tracking, and delivery operations.

The main entities in the domain include **Order, Shipment, Warehouse, Driver, Handoff, DeliveryProof, ExceptionReport, and Notification**.

Order

The **Order** entity represents a customer request to transport goods from a pickup location to a delivery location.

An order may initially be created with incomplete information and later updated as additional details become available.

Key attributes include:

- Order ID
- Customer name
- Pickup address
- Delivery address
- Order status

Each order can contain one or more shipments depending on operational requirements.

Shipment

The **Shipment** entity represents the physical movement of goods through the logistics network.

Shipments move through different operational states such as pickup, warehouse transfer, out-for-delivery, and delivered.

Key attributes include:

- Shipment ID
- Origin location
- Destination location
- Shipment status
- Assigned driver

A shipment may pass through multiple warehouses before reaching its final destination.

Warehouse

The **Warehouse** entity represents logistics hubs where shipments are temporarily stored during transportation.

Warehouses are responsible for confirming the arrival and dispatch of shipments.

Key attributes include:

- Warehouse ID
- Location
- Storage capacity

Driver

The **Driver** entity represents logistics personnel responsible for picking up and delivering shipments.

Drivers interact with the system primarily through mobile devices.

Key attributes include:

- Driver ID
- Driver name
- Vehicle number

Drivers confirm pickup and delivery events within the system.

Handoff

The **Handoff** entity records custody transfers of shipments between logistics personnel or operational units.

This entity provides traceability of shipment ownership during its lifecycle.

Key attributes include:

- Handoff ID
- Timestamp
- From entity
- To entity

DeliveryProof

The **DeliveryProof** entity represents verification of successful shipment delivery.

Proof may include photos, digital signatures, or other forms of confirmation captured by drivers.

Key attributes include:

- Proof ID
- Image or signature reference
- Upload timestamp

Delivery proof is linked to shipments and used for billing verification.

ExceptionReport

The **ExceptionReport** entity captures operational problems that occur during shipment processing.

Examples include:

- Shipment delays
- Damaged goods
- Partial deliveries
- Failed deliveries

Recording these events helps operations teams track and resolve issues efficiently.

Notification

The **Notification** entity represents system-generated messages sent to customers or internal staff when important shipment events occur.

Examples include pickup confirmations, delay alerts, and delivery confirmations.

Information Model

The information model defines how logistics data is structured and managed within the system. It describes the relationships between core entities and the types of information stored in the platform.

Data Relationships

The relationships between the main entities are as follows:

- One **Order** may contain multiple **Shipments**.
- A **Shipment** may pass through multiple **Warehouses**.
- A **Shipment** may have multiple **Handoff** records representing custody transfers.
- A **Shipment** may generate multiple **ExceptionReports** during its lifecycle.
- A **Shipment** may have one **DeliveryProof** after successful delivery.
- **Notifications** are generated when shipment status changes.

Data Categories

The system stores several categories of information:

Operational Data

Operational data includes information related to orders, shipments, warehouse transfers, and logistics events.

User Data

User data contains information about system users such as operations staff, drivers, finance personnel, and management.

Media Data

Media data includes delivery proof files such as photos or signatures uploaded by drivers during delivery confirmation.

Audit Data

Audit data includes event history such as shipment status updates, handoff records, and exception reports. This data supports traceability and accountability.

Role-Based Access Control (RBAC) Model

The Logistics Operations Platform implements **Role-Based Access Control (RBAC)** to ensure that system users can only perform actions permitted for their roles.

Each user is assigned a role, and permissions are granted based on that role.

Sales Manager

Sales managers are responsible for creating and updating customer orders.

Permissions include:

- Create orders
- Modify order details

Operations Coordinator

Operations coordinators manage shipment assignments and logistics operations.

Permissions include:

- Assign shipments to drivers or warehouses
- Update shipment status
- Record operational exceptions
- Monitor shipment progress

Warehouse Supervisor

Warehouse supervisors confirm the movement of goods within warehouse facilities.

Permissions include:

- Confirm inbound shipments
- Confirm outbound shipments
- Record warehouse transfer events

Driver

Drivers interact with the system through mobile devices.

Permissions include:

- Confirm shipment pickups
- Confirm shipment deliveries
- Upload delivery proof

Customer Support

Customer support staff assist customers by providing shipment status information.

Permissions include:

- Search shipment records
- View shipment status

These users have read-only access.

Finance and Billing

Finance staff use the system after delivery completion.

Permissions include:

- View shipment completion records
- Verify delivery proof
- Generate invoices

Management

Management users monitor operational performance.

Permissions include:

- View dashboards
- Access operational reports

System Architecture Overview

The Logistics Operations Platform follows a **layered service-oriented architecture** that separates user interfaces, system services, and data storage.

Presentation Layer

The presentation layer provides user interfaces for interacting with the system.

Examples include:

- Web portal used by operations staff, finance teams, and management
- Mobile interface used by drivers and warehouse personnel

Application Layer

The application layer exposes APIs that allow client applications to communicate with backend services.

Responsibilities include request routing, authentication, and input validation.

Service Layer

The service layer implements the core business functionality of the system.

Key services include:

- Order Management Service
- Shipment Tracking Service
- Exception Management Service
- Notification Service
- Billing and Reporting Service

Each service manages a specific domain of system functionality.

Data Layer

The data layer stores persistent system information including orders, shipment history, operational events, and delivery proof files.

This separation ensures reliable storage and supports reporting and auditing.

Subsystem Responsibilities

The system is divided into several subsystems, each responsible for a specific set of operations.

Order Management Subsystem

Handles order creation, updates, and order lifecycle management.

Shipment Tracking Subsystem

Tracks shipment movement and maintains shipment status throughout the logistics lifecycle.

Exception Management Subsystem

Records operational problems such as delays, damages, and failed deliveries.

Notification Subsystem

Sends automated notifications to customers and internal stakeholders when shipment events occur.

Billing and Reporting Subsystem

Generates shipment summaries and financial records required for billing and management reporting.

Interfaces Between Subsystems

Subsystems interact with each other through defined interfaces.

For example:

- The **Order Management subsystem** provides order information to the Shipment Tracking subsystem.
- The **Shipment Tracking subsystem** generates shipment status events that trigger notifications.
- Delivery confirmation events are used by the **Billing subsystem** to generate invoices.
- Exception records are stored by the **Exception Management subsystem** and displayed to operations staff.

These interfaces allow system components to communicate while maintaining loose coupling between subsystems.

Design Rationale

The system architecture is designed to support scalability, maintainability, and operational transparency.

Separating the system into independent subsystems improves modularity and allows each component to evolve independently.

Mobile support enables drivers and warehouse staff to record logistics events quickly with minimal interaction.

Recording shipment events and delivery proof creates a reliable audit trail that supports dispute resolution and billing verification.

Assumptions and Risks

Assumptions

Drivers will have smartphones capable of accessing the mobile interface.

Warehouse facilities will have internet connectivity for system access.

Customers will receive shipment updates through messaging services.

Risks

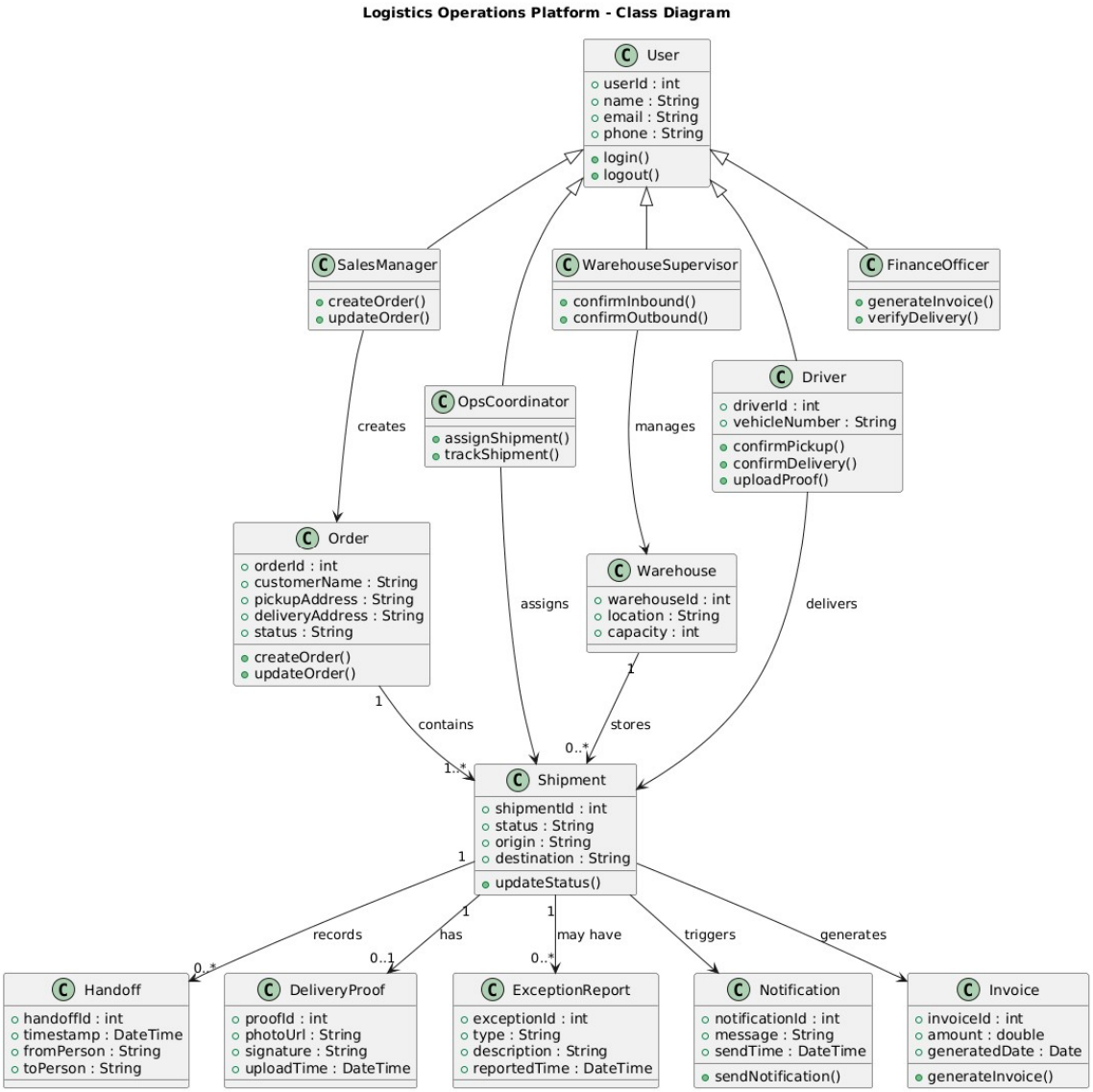
Network connectivity issues may delay updates from drivers.

Large delivery proof files may increase storage requirements.

External messaging services may introduce reliability constraints.

These risks can be mitigated through offline data caching, media compression, and retry mechanisms.

UML CLASS DIAGRAM



Class Diagram Description

The class diagram represents the static structure of the Logistics Operations Platform. It shows the main system entities, their attributes, operations, and relationships.

The **User** class represents all system users and is extended by specialized roles such as SalesManager, OpsCoordinator, WarehouseSupervisor, Driver, and FinanceOfficer.

The **Order** class represents customer orders created by sales staff. Each order may contain one or more shipments.

The **Shipment** class is the core entity that tracks logistics movement between pickup and delivery locations.

The **Warehouse** class represents logistics hubs where shipments are stored temporarily.

The **Handoff** class records custody changes between personnel such as warehouse staff and drivers.

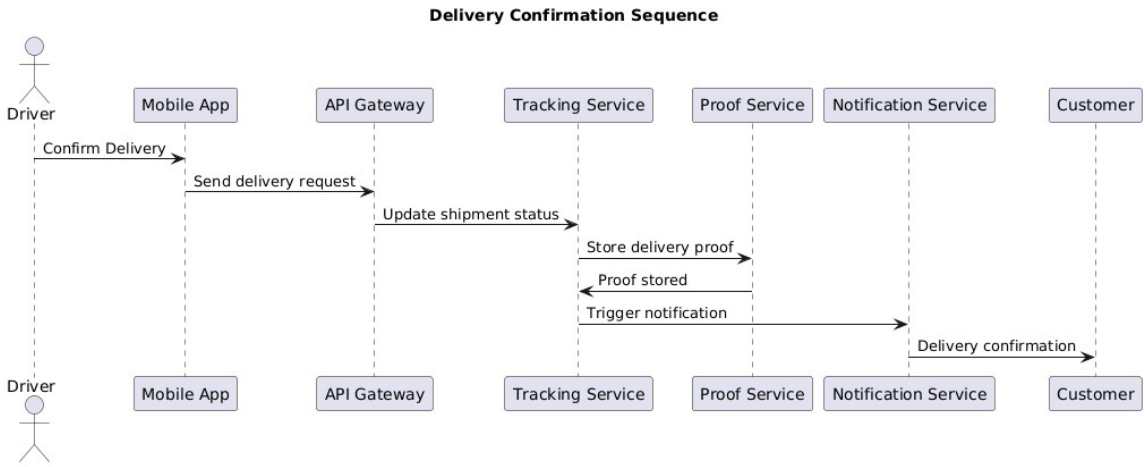
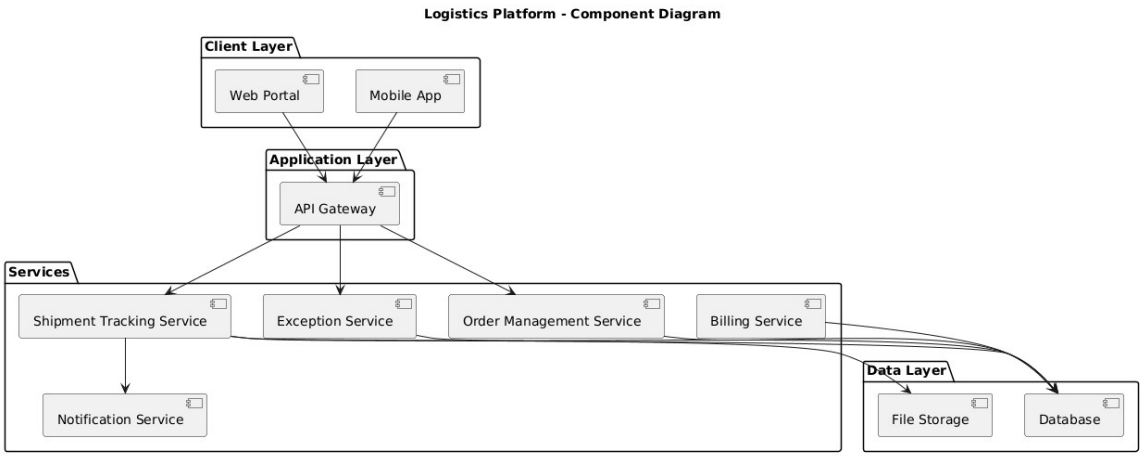
The **DeliveryProof** class stores delivery verification such as photos or signatures uploaded by drivers.

The **ExceptionReport** class records operational issues such as delays, damages, or failed deliveries.

The **Notification** class is responsible for sending updates to customers during important shipment stages.

The **Invoice** class supports billing operations by generating financial records after delivery completion.

Sequence diagram:



Component diagram: